Combinatorics II

Kon Yi

March 20, 2026

Abstract

Lecture note of Combinatorics II.

Contents

Chapter 1

Sorting Algorithm

Lecture 1

Problem 1.0.1. Given n distinct numbers $a_1, a_2, \ldots, a_n$, sort them in increasing order.

- Input: n distinct numbers $a_1, a_2, \ldots, a_n$.
- Output: a permutation $\pi \in S_n$ s.t.

$$a_{\pi(1)} < a_{\pi(2)} < \cdots < a_{\pi(n)}.$$

Observation. Given x, y, is $x < y$?

Remark 1.0.1. When analyzing an algorithm, we consider the worst-case input and count operations, where operations are defined according to context.

Algorithm 1.1: Brute force

Go through all $\pi \in S_n$ one by one and test each permutation to see if it is correct.

Remark 1.0.2. For running time, testing a permutation takes $\leq n-1$ comparisons, and there are $n!$ possible permutations, so

$$\# \text{ of comparisons} \leq n!(n-1).$$

Question. What is efficient? How does the running time scale as the input size n increases? Ideally, the running time should be polynomial $O(n^c)$ for some $c \in \mathbb{R}$.

Algorithm 1.2: Insertion Sort

Maintain a sorted partial list, add one element at a time until the entire list is sorted.

Algorithm 1.3: Binary insertion sort

```
// As before, we maintain a sorted prefix.
```

Insert a_i into sorted list $a_1 < a_2 < \cdots < a_{i-2} < a_{i-1}$. Compare a_i to the middle element. If similar, insert into the first half. If bigger, insert into the second half. `// Either way reduce number of possible positions by half each time`

Remark 1.0.3. Thus, we can insert in $\approx \log_2 i$ many comparisons each time, and thus in total we need $O(n \log_2 n)$ comparisons.

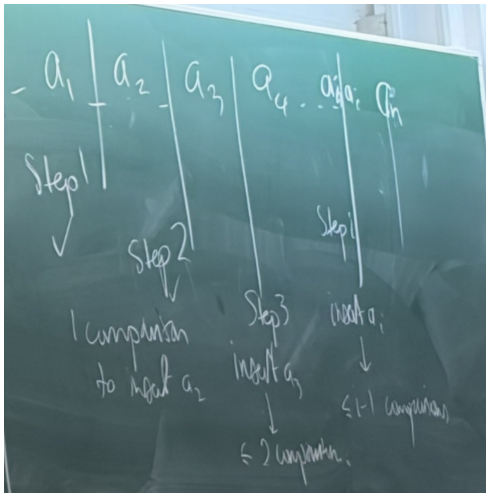
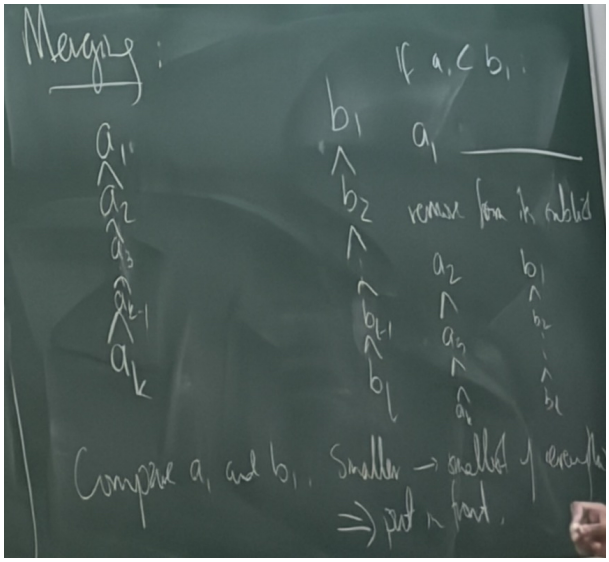


Figure 1.1: Insertion sort

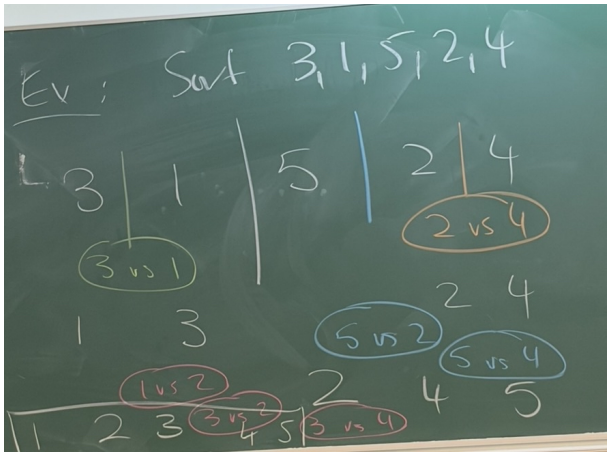
Algorithm 1.4: Merge Sort

```
// Recursive algorithm
```
Split the list into two equal halves.
Sort each half.
Merge the two sorted lists into one. `// Compare the head of two list, remove the smaller one from original list then put it in the back of a new list, which is empty initially, and repeat this comparison step.`

Remark 1.0.4. Number of comparison to merge $\leq n - 1$.



(a) Merging



(b) An example of merge sort

Remark 1.0.5. Running time: Let $T(n)$ be the worst case running time for merge sort on a list of

n numbers. Then $T(1) = 0$, and we know

$$T(n) \leq \underbrace{T\left(\lfloor \frac{n}{2} \rfloor\right)}_{\substack{\text{sorting the} \\ \text{first half}}} + \underbrace{T\left(\lceil \frac{n}{2} \rceil\right)}_{\substack{\text{sorting the} \\ \text{second half}}} + \underbrace{n-1}_{\text{merge}}.$$

Now suppose $n = 2^k$, $k \in \mathbb{N}$, then

$$\begin{cases} T(1) = 0 \\ T\left(2^k\right) = 2T\left(2^{k-1}\right) + 2^k - 1 \text{ for } k \geq 1. \end{cases}$$

Thus,

$$\begin{aligned} T\left(2^k\right) &= 2T\left(2^{k-1}\right) + 2^k - 1 \\ &= 2\left[2T\left(2^{k-2}\right) + 2^{k-1} - 1\right] + 2^k - 1 \\ &= \cdots = 2^i T\left(2^{k-i}\right) + i2^k - \left(2^i - 1\right), \end{aligned}$$

and if we let $i = k$, then

$$T\left(2^k\right) = 2^k T(1) + k2^k - 2^k + 1 = k2^k - 2^k + 1 = n \log_2 n - n + 1.$$

Thus, $T(n) \leq n \log_2 n - n + 1$ when $n = 2^k$.

Question (Lower bounds). Can we do better?

Answer. No, but to prove this, we need to show that any other algorithm, no matter how weird, takes as long. ⊛

Question. Suppose we have an algorithm A for sorting n numbers. Now can we lower-bound its runtime?

Information Theory approach

Each comparison gives ≤ 1 bit of information. At the end, we get the information of the permutation to sort the numbers, and since there are $n!$ possible permutations, so the number of time needed is $\geq \log_2 n!$.

Lecture 2

An arbitrary sorting algorithm is a function that takes the sequence of answers to our queries (is $x_i < x_j$?) and returns a permutation $\pi \in S_n$ s.t. $x_{\pi(1)} < x_{\pi(2)} < \cdots < x_{\pi(n)}$. Suppose the algorithm has a worst-case running time of k. Then this function has at most 2^k different outputs since such binary decision tree can has at most 2^k leaves and each leaf represent a possible permutation.

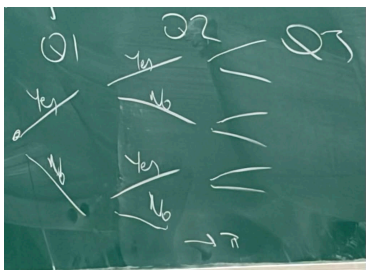


Figure 1.3: The decision tree

The entirety of S_n has to be in the image, so $2^k \geq |S_n| = n!$. Hence, $k \geq \log_2(n!) = (1 + o(1))n \log_2 n$.

Alternative arguments (playing against the devil)

You ask the comparisons to the devil, whose goal is to make the game go on as long as possible, The devil maintains a list of all possible permutations. Every time we do a comparison, we split the list into two: Those whose the answer is "yes" and those where it is "no". The devil chooses the answer with the bigger part. Game ends when the list has size 1. Let L_i be the list of possible permutations after i comparisons. $L_0 = S_n$, and for all $i \geq 1$,

$$|L_i| \geq \frac{1}{2}|L_{i-1}|.$$

Suppose one algorithm requires k comparisons in the worst case, then

$$1 \geq |L_k| \geq \frac{1}{2}|L_{k-1}| \geq \frac{1}{4}|L_{k-2}| \geq \cdots \geq \frac{1}{2^k}|L_0| = \frac{n!}{2^k},$$

so $2^k \geq n!$, which shows $k \geq (1 + o(1))n \log_2 n$.

Chapter 2

Connectivity and Spanning Tree

Definition 2.0.1 (graph). A (simple) graph is an ordered pair of sets (V, E), where V is a set of vertices and $E \subseteq \binom{V}{2}$ is a set of edges, where an edge is an unordered pair of disjoint vertices.

Definition 2.0.2. A walk in a graph is a sequence

$$v_0 e_1 v_1 e_2 v_2 e_3 v_3 \dots e_n v_n$$

where $v_i \in V$ and $e_i = \{v_{i-1}, v_i\} \in E$ for all i. A trail is a walk that does not repeat any edges. A path is a trail/walk without repeated vertices. A circuit is a trail that starts and ends at the same vertex. A cycle is a circuit that doesn't repeat any internal vertex.

Definition 2.0.3. A graph $G = (V, E)$ is connected if for every $u, v \in V$, there is a path(walk) from u to v.

Definition 2.0.4. Given a graph $G = (V, E)$ and a vertex $v \in V$, the connected componnent of v is the maximal connected subgraph of G that contains v.

Remark 2.0.1. $G' = (V', E')$ is a subgraph of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E \cap \binom{V'}{2}$. It is induced if $E' = E \cap \binom{V'}{2}$. It is spanning if $V' = V$.

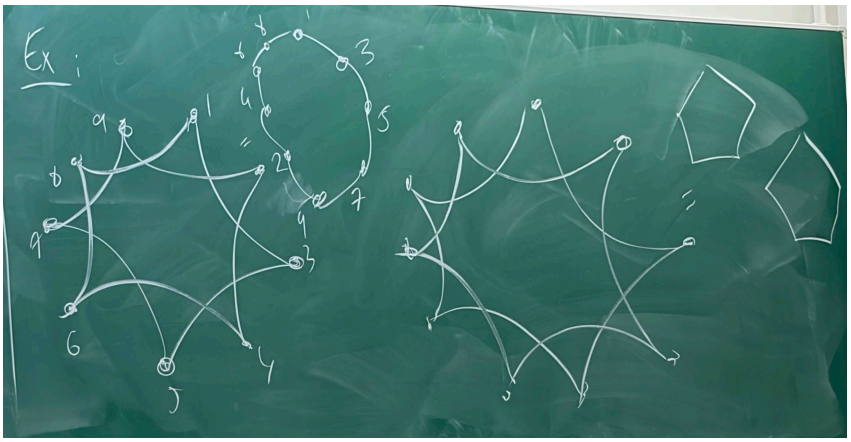


Figure 2.1: An example of connectivity (left one is connected but right one not)

Question. How can we identify the connected components of a graph?

Algorithm 2.1: Find the connected component containing v

Input: A graph $G = (V, E)$ and a vertex $v \in V$
Output: Connected component $C \subseteq V$ s.t. $v \in C$

```
// We will maintain two sets of vertices. Let W be the component so far, i.e. vertices we can reach from v whose neighborhood we have explored.
```

Initially $W_0 = \varnothing$. $Q_0 = \{v\}$. `// Q =queue of vertices to explore`
for *the i-th step* **do**
  if $Q_{i-1} = \varnothing$ **then**
    DONE. Return W as the component as v.
  else
    Let w be an arbitrary vertex in Q_{i-1}. $Q_i = Q_{i-1} \setminus \{w\}$, $W_i = W_{i-1} \cup \{w\}$.
    Explore the neighborhood of w: If u is a neighborhood of w and $u \notin Q \cup W_i$, then add it to Q_i.

Claim 2.0.1. The algorithm is finite.

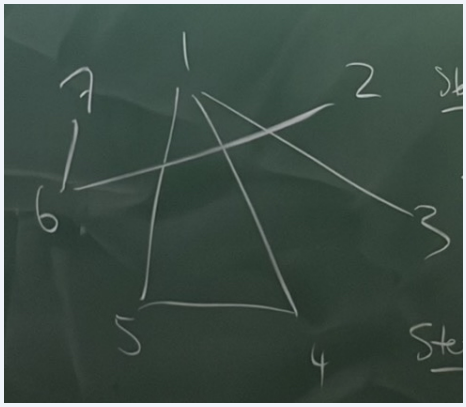
Proof. In every step, if we are not done, we add a distinct item to W (since Q_i and W_i are disjoint) and never remove it. Thus, we do at most $|V(G)|$ steps. ⊛

Claim 2.0.2. The algorithm is correct.

Proof. Let C_v be the component of v and W be the output of the algorithm. We show that $C_v = W$. We first show that $W \subseteq C_v$. Prove by induction on number of steps that we only add something to W if it was in Q. Also, we only add something to Q if it was a neighbor of an earlier member of W. By induction, there is a path from this vertex to v. Now we show $C_v \subseteq W$. Suppose not, then let $x \in C_v \setminus W$. Then there exists vx-path in G, say $v = v_0 v_1 v_2 \dots v_t = x$. Let j be the smallest index s.t. $v_j \notin W$. Then v_{j-1} is in W. Thus, we have explored its neighborhoods, so it would be in W. ⊛

It would be nice to not only know the components of v, but also paths that get there from v. Thus, in the algorithm, when exploring the neighborhood of a vertex $w \in Q_{i-1}$, if we add a neighbor u to Q_{i-1}, we keep track of the edge $\{w, u\}$ that was used to reach u.

Example 2.0.1. Now if $v = 4$ and the graph is the following figure:



Then we know

- Step 0: $Q_0 = \{4\}$, $W_0 = \{\}$.
- Step 1: $w = 4$, $Q_1 = \{1(\{1, 4\}), 5(\{4, 5\})\}$, and $W_1 = \{4\}$.
- Step 2: $w = 1, Q_2 = \{5(\{4, 5\}), 3(\{1, 3\})\}$, and $W_2 = \{4, 1(\{1, 4\})\}$.

- Step 3: $w = 5, Q_3 = \{3(\{1,3\})\}$, and $W_3 = \{4, 1(\{1,4\}), 5(\{4,5\})\}$.
- Step 4: $w = 3, Q_4 = \{\}$, and $W_4 = \{4, 1(\{1,4\}), 5(\{4,5\}), 3(\{1,3\})\}$.

Definition 2.0.5. A tree is a minimally connected graph, i.e. it is connected, but removing any edge disconnect it.

Proposition 2.0.1. Let T be an n-vertex graph. The following is equivalent:

(1) T is a tree.

(2) T is maximally acyclic.

(3) T is connected and has $n-1$ edges.

(4) T is acyclic and has $n-1$ edges.

Remark 2.0.2. Acyclic means no cycle.

Lecture 3

If we modify **??** so that we take the shortest element in Q, then we call this algorithm **Breadth-first search**.

If we modify **??** so that we take the newest element in Q, then we call this algorithm **Depth-first search**.

2.1 Minimum weight spanning tree

Theorem 2.1.1 (Cayley)**.** The complete graph K_n has n^{n-2} spanning trees.

We want to find the most efficient/cheapest spanning tree.

Problem 2.1.1 (Mininimum spanning tree)**.**

- Input: Complete graph K_n and a weight function $w : E(K_n) \to \mathbb{R}_{\geq 0}$
- Output: A spanning tree T of K_n minimizing $\sum_{e \in E(T)} w(e)$.

Example 2.1.1. If G is an n-vertex graph (not necessarily complete). Define

$$w(e) = \begin{cases} 0, & \text{if } e \in E(G); \\ 1, & \text{if } e \notin E(G). \end{cases}$$

Then G is connected if and only if there exists a tree of weight 0.

Algorithm 2.2: Kruskal's algorithm(Theoretical way)

```
// Maintain F, and let F = ∅ initially.
Order the edges by weight, say e_1, e_2, ..., e_m.
for each i s.t. 1 ≤ i ≤ m do
    if F + e_i contributes a cycle then
        do nothing.
    else
        add e_i to F.
```

Algorithm 2.3: Kruskal's algorithm(Implementation way)

```
// Maintain F, and let F = ∅ initially, and C_v is the component containing v
for all v ∈ V(G), where C_v = {v} initially for all v ∈ V(G).
Order the edges by weight, say e_1, e_2, ..., e_m.
for each i s.t. 1 ≤ i ≤ m do
    Let e_i = (u, v). if C_u = C_v then
        do nothing.
    else
        add e_i to F.
        Merge C_u and C_v.
```

Claim 2.1.1. At the end of the Kruskal's algorithm, F is a minimum spanning tree.

Proof. We have to show F is a spanning tree and F is minimal.

We first show F is a spanning tree. It suffices to show F has no cycles and connected. Note that in this algorithm we make sure every adds of an edge into F would not contribute a cycle, so the final F must has no cycle. Now we show F is connected. If not, then we have connected components. Since G is connected, there will be edges between components. Thus the algorithm would have added the first edge between these components. Now we know F is connected and acyclic. If F does not cover all vertices, say not covering v, then the edge between v and F would be added in the algorithm, so F covers all n vertcies, i.e. F is a spanning tree.

Now let T_k be the tree from Kruskal's algorithm. Let $T_{\min}$ be a minimum spanning tree that maximizes $|E(T_k) \cap E(T_{\min})|$. If $|E(T_k) \cap E(T_{\min})| = n - 1$, then T_k is a minimum spanning tree. Otherwise, consider the first edge (i.e. e_v with minimal v) in $T_k \Delta T_{\min}$, say e_i.

Claim 2.1.2. $e_i \in T_k$.

Proof. If $e_i \in T_{\min}$, then in the i-th step of Kruskal algorithm, we would have added e_i to T_k. (For $e_1, e_2, \ldots, e_{i-1}$, each edge is either in both tree or not in any of both, so e_i can be added to T_k in the algorithm since it will not contribute to a cycle (since $T_{\min}$ has no cycle)). Thus, $e_i \in T_k$. ⊛

Consider $T_{\min} + e_i$, which creates a cycle C. Thus, C must contain an edge e_j with $j > i$ (because all edges e_j with $j < i$ are common to both trees, and $C \nsubseteq T_k$). Let $T' = T_{\min} + e_i - e_j$, then T' is a spanning tree with smaller weight sum, which is a contradiction. ⊛

Lecture 4

Running time of Kruskal's algorithm

First, sorting all edges by weight takes $O(m \log_2 m)$ steps. Then, iterate through all edges, if $c_u = c_v$, do nothing, and this steps take $O(1)$ steps. If not, then add edges and update components, which takes $O(n)$ steps, and will run this case $n - 1$ times. Thus, the total time:

$$O(m \log_2 m) + O(m) + O\left(n^2\right) = O(m \log_2 m + n^2) = \begin{cases} O(m \log m), & \text{if } m = \Omega\left(\dfrac{n^2}{\log n}\right); \\ O\left(n^2\right), & \text{if } O\left(\dfrac{n^2}{\log n}\right). \end{cases}$$

With more improvement,

$$O(m \log m) \text{ for all } m.$$

Chapter 3

Dijkstra's Algorithm

Drawbacks of trees as networks

(1) Unique path between any two vertices, which could be unnecessarily long.

(2) No redundancy/robustness: removing any one edge/vertex disconnects the network.

Question. Given a network $G = (V, E)$ and weights $w : E \to \mathbb{R}_{\geq 0} \cup \{\infty\}$, and given two vertices $u, v \in V$, how do we find the cheapest path from u to v?

Observation. We may assume G is complete. If an edge $\{x, y\}$ is missing, add it with ∞ weight.

Lemma 3.0.1. If P is a cheapest path from u to v, then for any $x, y \in P$, $P[x, y]$ is a shortest path from x to y.

Algorithm 3.1: Dijkstra's algorithm

Data: A complete graph $G = (V, E)$.
Result: For any vertex x, the cheapest graph from u to x.
Idea: Keep a set W of vertices which we have already found the cheapest path from u. For other vertices, keep track of a tentative cheapest path, i.e. the cheapest path we have found so far.

```
W ← {u}, t(u) = 0, prev(u) = ∅.
For any x ∈ V \ {u}, t(x) = w({u, x}), prev(x) = u.
while W ≠ V do
    Let y = arg min_{x∈V\W} t(x), i.e. t(y) minimizes t(x) for all x ∉ W.
    Add y to W.
    for every x ∉ W ∪ {y} do
        if t(x) > t(y) + w({y, x}) then
            t(x) ← t(y) + w({y, x}), prev(x) = y.
Cheapest path from x to u is x, prev(x), prev(prev(x)), ..., u with cost t(x).
```

Theorem 3.0.1. Dijkstra's algorithm terminates after $n - 1$ rounds and returns shortest path.

Proof. Each round adds a new vertex to W, which never gets removed, so after $n - 1$ rounds, $W = V$. For correctness, we can do induction on the number of rounds:

Claim 3.0.1. At the beginning of the i-th iterartion:

(a) $|W| = i$.

(b) for every $x \in V$, the vertices $x, \text{prev}(x), \text{prev}(\text{prev}(x)), \dots, u$ form a path from x to u of weight $t(x)$, where all vertices except x lie in W.

(c) for any vertex $z \in V \setminus W$ with $t(z) = \min\{t(x) : x \in V \setminus W\}$, $t(z)$ is the minimum cost of a uz-path.

(d) For every $y \in V \setminus W$ and $v \in W$,

$$t(y) \leq t(v) + w(\{v, y\}).$$

Proof.

- Base case: $i = 1$, (a), (b), (d) are obvious. For (c), suppose not, then if z minimize $t(\cdot)$ but there is a cheaper uz-path, say

$$u \to x \to \cdots \to z \in P,$$

then

$$w(P) = w(\{u, x\}) + w(\{x, \dots\}) + \cdots \geq w(\{u, x\}) = t(x) \geq t(z),$$

which is a contradiction.

- Induction step: (a) is obvious.

Now we show (b). If we didn't update $t(x), \text{prev}(x)$ in the $(i-1)$-th iteration, then this is true by induction. If we did update, then $\text{prev}(x) = v_0$ where v_0 is the vertex added to W in the $(i-1)$-th iteration, and $t(x) = t(v_0) + w(\{v_0, x\}) =$ weight of path $u \to \cdots \to v_0 \to x$, and u to $\text{prev}(v_0)$ all in W by induction, and v_0 was added to W in the $(i-1)$-th iteration.

Now we show (c). Suppose not, then $\exists z \in V \setminus W$ minimizing $t(z)$ but $t(z) > d(z, u)$. Consider the cheapest path from u to z, say P. Let y be the first vertex in P (starting from u) that is not in W. (such y exists since z is not in W) Let x be the preceding vertex on P, noting that $x \in W$. Then,

$$t(z) > w(P) = w(P_{[u,x]}) + w(\{x, y\}) + w(P_{[y,z]}) \geq w(P_{[u,x]}) + w(\{x, y\}).$$

By **??**, $P_{[u,x]}$ has to be a lightest path from u to x. By induction (where we add x to W), $t(x) = w(P_{[u,x]})$. Thus,

$$t(z) > w(P_{[u,x]}) + w(\{x, y\}) = t(x) + w(\{x, y\}) \geq t(y),$$

which is by the induction statement of (d). However, this contradicts z minimizeing $t(v)$ for $v \in V \setminus W$.

As for (d), after the step where we added v to W, we check whether $t(v) + w(\{v, y\}) < t(y)$, and update if so. Hence, after this round, $t(v)$ will not change, and $t(v)$ is decreasing, so it is impossible that $t(y) > t(v) + w(\{v, y\})$.

⊛

This implies the correctness: for any x, at the iteration when we add x to W, (c) implies we have the correct path for x and we never change it subsequently. ■

Lecture 5

3.1 Running Time of Dijkstra's Algorithm

Since we have $n-1$ iterations, and in each iteration, we have to find the minimum, which takes $O(n)$ steps, and updating also takes $O(n)$ steps, so the total running time is $O\left(n^2\right)$.

3.2 Traveling Salesman Problem

Problem 3.2.1 (Traveling Salesman Problem)**.** If we have a weighted graph $G = (V, E)$, where each vertex is a city and each edge is a road between two cities, and we are at $v \in V$ initially. Then, how to travel to every city exactly once and then return to the starting point v while minimizin the cost?

Definition 3.2.1 (Hamilton cycle)**.** A cycle that goes through every vertex in a graph is called a Hamilton cycle.

Remark 3.2.1. Traveling Salesman Problem's goal is to find a Hamilton cycle of minimum cost of the given graph.

Observation. The number of possible routes is $\frac{1}{2}(n-1)!$ since we can just permutate the $n-1$ vertices that is not the starting point, and note that the direction is not important, so $u \to a \to b \to u$ and $u \to b \to a \to u$ is considered same routes.

Thus, the brute force takes $O(n) \cdot \frac{1}{2}(n-1)! = O(n!)$ time since for each route we have to go through this path to compute the cost.

Remark 3.2.2. The Held-Karp Algorithm, which uses dynamic programming, takes $O\left(n^2 2^n\right)$ time to solve Traveling Salesman Problem (also exponential in memory).

Question (Open question)**.**

- Is there a polynomial time algorithm?
- Is there an algorithm takes $O(1.99999999^n)$ time?

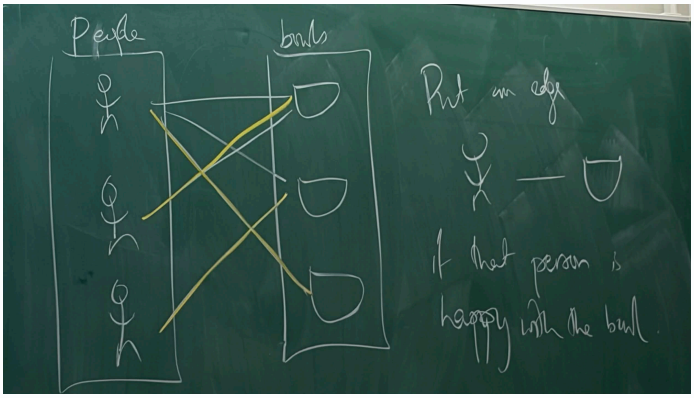
Chapter 4

Matchings

Lecture 6

Example 4.0.1. n friends going for dinner. n bowls of beef noodle soup served. Each person has a subset of the bowls they would be happy with. Can we give one bowl to each person and making every one happy?

Proof. We can build a graph as follow:



⊛

Definition 4.0.1. A graph G is bipartite if we can partition $V(G) = X \cup Y$ s.t. for every $e \in E(G)$,

$$|e \cap X| = 1 = |e \cap Y|.$$

Definition 4.0.2. A matching in a graph is a set of pairwise-disjoint edges. If a vertex is in an edge of the matching, it is called "saturated". If every vertex is saturated, the matching is perfect.

Observation. If there is some $S \subseteq X$ with $|N(S)| < |S|$, then there is no matching saturating X.

Proof. If there is a matching saturating X and $\exists S \subseteq X$ with $|N(S)| < |S|$, then each vertex in S would match with some vertex in $N(S)$ in the matching saturating X. Note that every vertex in S cannot match to same vertex. Hence, $|N(S)| \geq |S|$, which is a contradiction. ■

Theorem 4.0.1 (Hall's Marriage Theorem)**.** Let $G = (X \cup Y, E)$ be a bipartite graph. If $|N(S)| \geq |S|$ for all $S \subseteq X$, then G has a matching saturating X.

Proof. We can do induction on $|X|$.

- Base case: $|X| = 1$. Let $S = X$, and $|N(X)| \geq |X| = 1$, then there exists $y \in Y$ s.t. $\{x, y\} \in E$, so we can pick this edge to form a matching saturating X.
- Induction Step: $|X| \geq 2$.
  - Case 1: $\forall \varnothing \neq S \subsetneq X$, $|N(S)| > |S|$. Let $\{x, y\}$ be an arbitrary edge, where $x \in X$ and $y \in Y$. Let $G' = (X' \cup Y', E')$ where $X' = X - \{x\}$, $Y' = Y - \{y\}$, and $E' = E|_{X' \times Y'}$. We use induction to find a matching M' in G' saturating X'. Then, $M = M' \cup \{x, y\}$ is a matching saturating X. Note that we can use induction hypothesis since for $S \subseteq X'$, $N_{G'}(S) = N_G(S) \setminus \{y\}$. Hence,
    $$|N_{G'}(S)| \geq |N_G(S)| - 1 \geq |S|,$$
    and thus we can find M'.
  - Case 2: there exists some $\varnothing \neq X_1 \subsetneq X$ with $|N(X_1)| = |X_1|$. Let $N(X_1) = Y_1$. Let $X_2 = X \setminus X_1$, and $Y_2 = Y \setminus Y_1$, and $G_1 = G[X_1 \cup Y_1]$. By induction, G_1 has a matching M_1 saturating X_1 since for all $S \subseteq X_1$, we have $S \subseteq X$ and thus
    $$|N_G(S)| \geq |S|,$$
    and note that $N_{G_1}(S) = N_G(S)$ since $N_G(S) \subseteq N_G(X_1) = Y_1$. If we can find a matching M_2 in $G_2 = G[X_2 \cup Y_2]$, then $M = M_1 \cup M_2$ is a matching saturating X. Let $S \subseteq X_2$. Consider $S' = S \cup X_1$. By assumption, $|N_G(S')| \geq |S'|$. Note that $|S'| = |S| + |X_1|$. Hence,
    $$N_G(S') = N_G(S \cup X_1) = Y_1 \cup \underbrace{N_{G_2}(S)}_{\subseteq Y_2}.$$
    Note that $N_G(S') = Y_1 \cup N_{G_2}(S)$ is a disjoint union since for $N_G(S)$, it is either in Y_1 or not, if not, then it is in $N_{G_2}(S)$. This shows
    $$|S| + |X_1| = |S'| \leq |N_G(S')| = |Y_1| + |N_{G_2}(S)| = |X_1| + |N_{G_2}(S)|.$$
    Hence, $|N_{G_2}(S)| \geq |S|$ for all $S \subseteq X_2$. By induction, we can find a matching M_2 in G_2 saturating X_2.

■

Corollary 4.0.1. If G is a d-regular bipartite graph, then G has a perfect matching.

Proof. Given any $S \subseteq X$,
$$d|S| = \#\{\text{edges between } S, N(S)\} \leq d|N(S)|$$
since for vertices in $N(S)$, they may be adjacent to vertex not in S, so by Hall's theorem, there is a matching saturating X, which is a perfect matching since
$$d|X| = \#\{\text{edges between } X, Y\} = d|Y|,$$
and thus $|X| = |Y|$. ■

Question. What are necessary and sufficient conditions to have a perfect matching in general (i.e. non-bipartite) graph?

For necessary condition:

- n should be even.
- every vertex should has positive degree.